

MASTER PROMPT — Enterprise Multi-Company / Multi-Branch POS & Inventory Management System

ROLE

Act as a **Senior Enterprise Software Architect, Database Architect, UI/UX Designer, POS Domain Expert, and Full-Stack Developer.**

I want you to design and develop a **modern, production-ready, generic Point of Sale (POS), Sales, Inventory, Purchasing, Accounting, Customer/Supplier, and Multi-Branch Management System.**

The application must be designed as a reusable commercial-grade POS platform that can work for different types of businesses such as:

- Retail stores
- Supermarkets
- Pharmacies
- Electronics stores
- Garments
- Grocery
- Hardware
- Spare parts
- General trading
- Wholesale
- Distribution
- Restaurants/cafes with suitable extensions

Do not create a toy/demo application.

The architecture must be scalable, secure, maintainable, modular, and suitable for real-world business usage.

1. TECHNOLOGY STACK

Use the following preferred technology stack:

Frontend

- Next.js
- React

- TypeScript
- HTML5
- Tailwind CSS
- Responsive design
- Modern component architecture
- Reusable UI components
- Keyboard-friendly POS interface

Backend

Preferred:

- Node.js
- TypeScript
- REST API or well-structured API architecture

You may use:

- Next.js API routes/server actions where appropriate
- Or a separate Node.js backend if that provides a cleaner enterprise architecture

Database

The system must be designed so that the database layer can support:

- PostgreSQL
- SQL Server
- Oracle

PostgreSQL can be the primary implementation database.

However, avoid unnecessarily using PostgreSQL-specific functionality throughout the business/domain layer so that SQL Server or Oracle can be supported later.

Use an ORM/query layer where appropriate.

Possible choices:

- Prisma
- Drizzle
- TypeORM
- another enterprise-quality ORM

Explain your choice.

Authentication

Implement secure authentication with:

- Username/email
- Password
- Password hashing
- Session/token management
- Role-based authorization
- Permission-based authorization
- Branch-level authorization
- Company-level authorization

Never store plain-text passwords.

2. APPLICATION OBJECTIVE

Build a complete ERP-style POS application with the following major areas:

1. Authentication
2. Dashboard
3. Company Management
4. Branch Management
5. User Management
6. Roles
7. Permissions
8. User Rights
9. Menu Management
10. Item/Product Management
11. Categories
12. Brands
13. Units
14. Barcodes
15. Inventory
16. Stock Transactions
17. Stock Transfers
18. Purchasing
19. Suppliers
20. Customers
21. Sales
22. POS Billing
23. Invoices
24. Cash Sales
25. Credit Sales
26. Returns

27. Discounts
 28. Customer-specific pricing
 29. Wholesale/retail pricing
 30. Reorder Level Management
 31. Accounting
 32. Chart of Accounts
 33. General Ledger
 34. Accounts Receivable
 35. Accounts Payable
 36. Cash Management
 37. Bank Management
 38. Payments
 39. Receipts
 40. Expenses
 41. Journal Entries
 42. Tax/VAT support
 43. Reports
 44. Audit Trail
 45. System Configuration
 46. Hardware/integration support
 47. Multi-company
 48. Multi-branch
 49. Branch-wise reporting
 50. Company-wise reporting
 51. Consolidated reporting
-

3. MULTI-COMPANY ARCHITECTURE

The system must support multiple companies.

Example:

Company A

- Head Office
- Branch 1
- Branch 2
- Branch 3

Company B

- Head Office
- Branch 1
- Branch 2

A company should have:

- Company ID
- Company code
- Company name
- Legal name
- Address
- Phone
- Email
- Tax/VAT registration
- Currency
- Fiscal year
- Logo
- Default settings
- Accounting configuration
- Inventory configuration
- POS configuration
- Invoice configuration

Users must be assigned to one or more companies depending on permissions.

4. MULTI-BRANCH ARCHITECTURE

Each company can have multiple branches.

Branch information:

- Branch ID
- Branch code
- Branch name
- Company ID
- Address
- Contact information
- Warehouse
- Default cash account
- Default bank account
- Default receivable account
- Default payable account
- Tax configuration
- POS configuration
- Invoice numbering
- Stock configuration
- Active/inactive

The system must support:

Branch-specific operations

A user may be restricted to:

- One branch
- Multiple branches
- All branches

The system should prevent unauthorized users from viewing or modifying data belonging to unauthorized branches.

5. DATA VISIBILITY

Every important transaction should support:

Company level

View data for a selected company.

Branch level

View data for:

- Current branch
- Selected branch
- Multiple branches
- All branches

Consolidated level

Allow authorized administrators to see:

- Overall stock
- Overall sales
- Overall purchases
- Overall receivables
- Overall payables
- Overall profit
- Overall cash
- Overall accounting transactions

Example:

Stock:

Company A

- Branch 1: 100 units
- Branch 2: 150 units
- Branch 3: 200 units

Overall Company A Stock = 450 units

The same principle should apply to financial and sales reporting.

6. AUTHENTICATION

Create a professional login system.

Features:

- Login
- Logout
- Forgot password
- Reset password
- Change password
- Remember session
- Session timeout
- Failed login tracking
- Account lockout
- Active/inactive user
- Password policy
- Optional 2FA architecture
- Login audit trail

After login, determine:

- User
 - Company
 - Branch
 - Role
 - Permissions
 - Default warehouse
 - Default POS terminal
 - Default price level
-

7. USER MANAGEMENT

Create complete user administration.

Fields:

- User ID
- Username
- Full name
- Email
- Phone
- Password
- Employee information
- Company
- Branch
- Role
- Status
- Last login
- Password expiry
- Created date
- Updated date

Support:

- Multiple roles
 - Multiple branches
 - User-specific permissions
 - User overrides
 - Login restrictions
-

8. ROLE & PERMISSION SYSTEM

Create a powerful RBAC system.

Permission examples:

Sales

- View sales
- Create sale
- Edit sale
- Delete sale
- Void sale
- Return sale

- Approve discount
- Approve price override
- Reprint invoice
- Cancel invoice

Inventory

- View stock
- Adjust stock
- Transfer stock
- Approve stock adjustment
- View valuation

Purchasing

- Create purchase
- Edit purchase
- Approve purchase
- Cancel purchase
- Purchase return

Accounting

- View accounts
- Create journal
- Post journal
- Reverse journal
- View ledger

Administration

- Manage users
- Manage roles
- Manage company
- Manage branches
- Manage configuration

Permissions should support:

- Create
- Read
- Update
- Delete
- Approve
- Export
- Print
- Void
- Post
- Reverse

9. MENU MANAGEMENT

Create a dynamic menu system.

Menu structure should be database-driven.

Example:

Dashboard

Sales

- POS
- Sales Invoice
- Sales Return
- Sales History
- Customer Receipts

Inventory

- Items
- Categories
- Brands
- Stock
- Stock Adjustment
- Stock Transfer
- Stock Ledger
- Reorder Report

Purchasing

- Purchase Invoice
- Purchase Return
- Suppliers
- Purchase History

Accounts

- Chart of Accounts
- Journal
- General Ledger
- Cash
- Bank
- Receivables
- Payables

Reports

Administration

The menu shown to a user must depend on permissions.

10. PRODUCT / ITEM MANAGEMENT

Create a comprehensive item master.

Fields should include:

- Item ID
- SKU
- Item code
- Item name
- Short name
- Description
- Category
- Subcategory
- Brand
- Manufacturer
- Model
- Unit
- Purchase unit
- Sales unit
- Conversion factor
- Barcode
- Multiple barcodes
- Cost price
- Average cost
- Last purchase price
- Retail price
- Wholesale price
- Minimum price
- Customer-specific price
- Tax
- Discount
- Reorder level
- Reorder quantity
- Maximum stock
- Minimum stock
- Opening stock
- Stock tracking
- Batch tracking

- Serial number tracking
 - Expiry tracking
 - Weight
 - Dimensions
 - Image
 - Active/inactive
-

11. MULTIPLE UNITS

Support units such as:

- Piece
- Box
- Carton
- Dozen
- Kg
- Gram
- Liter
- Meter

Example:

1 Carton = 24 Pieces

The POS should allow selling by:

- Carton
- Piece

Inventory must correctly calculate stock based on conversion rules.

12. BARCODE MANAGEMENT

Implement barcode functionality.

Support:

- Barcode generation
- Barcode printing
- Barcode scanning
- Multiple barcodes per item
- EAN-13
- EAN-8

- UPC
- Code 128
- Internal/custom barcode

Provide barcode label printing.

Allow users to generate barcodes for:

- New products
 - Existing products
 - Selected products
 - Bulk products
-

13. POS SCREEN

The POS screen is one of the most important parts of the application.

Create a fast, modern, keyboard-friendly interface.

Layout:

Left / Main area

Product search

- Barcode input
- Item search
- Category filters
- Brand filters
- Product tiles
- Favorites

Center

Cart:

- Item
- Barcode
- Quantity
- Unit
- Price
- Discount
- Tax
- Net amount

Right

Totals:

- Gross amount
- Discount
- Tax
- Net amount
- Paid
- Balance
- Change

Buttons:

- Cash
- Credit
- Card
- Bank
- Split payment
- Hold
- Recall
- Void
- Return
- Discount
- Price override
- Customer
- Print
- Email
- Save

14. POS KEYBOARD SHORTCUTS

Design keyboard shortcuts for fast cashier operation.

Example:

F1 = Search Item

F2 = Customer

F3 = Hold Sale

F4 = Recall Sale

F5 = Payment

F6 = Discount

F7 = Price Override

F8 = Return

F9 = Print

F10 = Complete Sale

ESC = Close dialog

ENTER = Confirm

Allow configurable shortcuts.

15. SALES TYPES

Support:

- Cash Sale
- Credit Sale
- Card Sale
- Bank Sale
- Split Payment
- Customer Account Sale
- Wholesale Sale
- Retail Sale
- Counter Sale
- Online Sale
- Delivery Sale

The architecture should allow additional sale types later.

16. PAYMENT MANAGEMENT

Support:

- Cash
- Credit Card
- Debit Card
- Bank Transfer

- Mobile Wallet
- Cheque
- Account
- Multiple payment methods

Example:

Invoice = 10,000

Cash = 4,000

Card = 3,000

Bank = 3,000

Total = 10,000

Implement proper payment allocation.

17. CUSTOMER MANAGEMENT

Customer master:

- Customer code
- Customer name
- Company
- Contact
- Address
- Phone
- Email
- Tax number
- Credit limit
- Credit days
- Price level
- Discount level
- Opening balance
- Status

Customer account should show:

- Invoices
- Payments
- Returns
- Outstanding balance
- Credit limit

- Aging
 - Ledger
-

18. CUSTOMER-SPECIFIC PRICING

Implement flexible pricing.

Support:

Standard price

Example:

Product A = 100

Wholesale price

Product A = 90

VIP price

Product A = 85

Customer-specific price

Customer ABC:

Product A = 82

Quantity-based price

1-9 = 100

10-49 = 95

50+ = 90

The pricing engine should determine the correct price automatically.

19. DISCOUNT ENGINE

Support:

- Item discount
- Invoice discount
- Percentage discount
- Fixed amount discount
- Customer discount
- Category discount
- Promotional discount
- Quantity discount
- Date-based discount
- Price-level discount

Support discount authorization.

Example:

Cashier can discount up to 5%.

Supervisor can discount up to 15%.

Manager can discount up to 30%.

Anything higher requires administrator approval.

20. SALES RETURNS

Support:

- Full invoice return
- Partial return
- Item return
- Quantity return
- Damaged return
- Customer return

Inventory and accounting must automatically reverse appropriately.

21. PURCHASING

Implement:

- Purchase order
- Goods receipt
- Purchase invoice
- Purchase return
- Supplier payment

Purchase invoice should support:

- Supplier
 - Items
 - Quantity
 - Cost
 - Discount
 - Tax
 - Freight
 - Other charges
 - Net amount
 - Payment
 - Balance
-

22. SUPPLIER MANAGEMENT

Supplier master:

- Supplier code
- Supplier name
- Contact
- Address
- Phone
- Email
- Tax number
- Credit limit
- Credit days
- Opening balance

Supplier ledger:

- Purchases
- Returns
- Payments
- Outstanding

- Aging
-

23. INVENTORY MANAGEMENT

Implement complete inventory management.

Stock transactions:

- Opening stock
- Purchase
- Purchase return
- Sale
- Sales return
- Stock adjustment
- Stock transfer
- Damage
- Expiry
- Consumption
- Stock write-off

Maintain a proper stock ledger.

Every inventory movement should have:

- Transaction ID
 - Date
 - Company
 - Branch
 - Warehouse
 - Item
 - Quantity
 - Unit
 - Cost
 - Reference document
 - User
 - Timestamp
-

24. STOCK TRANSFER

Support:

Branch A → Branch B

Warehouse A → Warehouse B

Transfer lifecycle:

1. Transfer request
2. Approval
3. Dispatch
4. Receive
5. Complete

Track:

- Sent quantity
 - Received quantity
 - Difference
 - Damaged quantity
-

25. REORDER LEVEL MANAGEMENT

Every product should support:

- Minimum stock
- Reorder level
- Reorder quantity
- Maximum stock

Dashboard alerts:

- Out of stock
- Below reorder level
- Overstock
- Expiring soon

Create reorder reports.

Optionally generate purchase suggestions automatically.

26. BATCH / EXPIRY MANAGEMENT

For businesses requiring it, support:

- Batch number
- Manufacturing date

- Expiry date
- Batch cost
- Batch quantity

Allow configurable inventory methods:

- FIFO
- FEFO
- Average Cost

FEFO should be available for expiry-sensitive businesses.

27. SERIAL NUMBER MANAGEMENT

For electronics and other serialized products:

- Serial number
- IMEI
- Warranty
- Purchase date
- Sale date
- Customer
- Supplier

Track serial number lifecycle.

28. ACCOUNTING MODULE

Create a proper double-entry accounting architecture.

Core modules:

- Chart of Accounts
- General Ledger
- Journal Entries
- Cash
- Bank
- Accounts Receivable
- Accounts Payable
- Expenses
- Income
- Assets
- Liabilities
- Equity

- Tax
-

29. CHART OF ACCOUNTS

Create hierarchical accounts.

Example:

1000 Assets

1100 Current Assets

1110 Cash

1120 Bank

1130 Accounts Receivable

1200 Inventory

2000 Liabilities

2100 Accounts Payable

3000 Equity

4000 Revenue

5000 Cost of Goods Sold

6000 Expenses

Support:

- Account code
 - Account name
 - Account type
 - Parent account
 - Company
 - Branch
 - Active/inactive
 - Posting/non-posting account
-

30. AUTOMATIC ACCOUNTING

Sales should automatically generate accounting entries.

Example:

Cash Sale:

Debit Cash

Credit Sales Revenue

And:

Debit Cost of Goods Sold

Credit Inventory

Credit sale:

Debit Accounts Receivable

Credit Sales Revenue

And:

Debit Cost of Goods Sold

Credit Inventory

Purchase:

Debit Inventory

Credit Accounts Payable

Payment:

Debit Accounts Payable

Credit Cash/Bank

Ensure all transactions remain balanced.

31. ACCOUNTING DIMENSIONS

Accounting transactions must support:

- Company
- Branch
- Warehouse
- Department
- Cost center
- Project if required

This allows reports such as:

Branch 1 Profit & Loss

Branch 2 Profit & Loss

Company-wide Profit & Loss

Consolidated Profit & Loss

32. CASH MANAGEMENT

Implement:

- Cash drawer
- Cash opening
- Cash closing
- Cash deposit
- Cash withdrawal
- Cash transfer
- Cash shortage
- Cash overage

Support POS shifts.

Example:

Opening Cash = 20,000

Sales Cash = 80,000

Expenses = 5,000

Expected Cash = 95,000

Actual Cash = 94,500

Difference = -500

Record cashier reconciliation.

33. POS SHIFT MANAGEMENT

Implement:

- Open shift
- Close shift
- Cashier
- POS terminal
- Opening cash
- Sales
- Returns
- Expenses
- Cash in/out
- Expected cash
- Actual cash
- Difference

Only authorized users can close/reopen shifts.

34. TAX MANAGEMENT

Create configurable tax engine.

Support:

- Tax-free
- Percentage tax
- Inclusive tax
- Exclusive tax
- Multiple taxes
- VAT
- Sales tax

Tax configuration should be company/branch configurable.

35. INVOICE MANAGEMENT

Invoices should have:

- Company information
- Branch information
- Invoice number
- Date/time
- Customer
- Cashier
- Items
- Quantity
- Price
- Discount
- Tax
- Total
- Payment
- Balance

Support:

- Thermal receipt
- A4 invoice
- Custom invoice template
- Reprint
- PDF
- Email

Invoice numbering should be configurable by:

- Company
- Branch
- Invoice type
- Financial year

36. PURCHASE / SALES NUMBERING

Implement configurable document numbering.

Examples:

INV-2026-000001

INV-B01-2026-000001

PUR-2026-000001

SR-2026-000001

PR-2026-000001

Allow prefixes and sequences to be configured.

Prevent duplicate numbers under concurrency.

37. REPORTING MODULE

Create a professional reporting system.

Sales Reports

- Daily sales
- Monthly sales
- Sales by branch
- Sales by company
- Sales by cashier
- Sales by customer
- Sales by item
- Sales by category
- Sales by brand
- Sales by payment method
- Sales by sale type
- Sales returns
- Discounts
- Tax

Inventory Reports

- Current stock
- Stock valuation
- Stock ledger
- Stock movement
- Stock adjustment
- Stock transfer
- Low stock
- Reorder
- Dead stock
- Fast-moving items
- Slow-moving items
- Batch report

- Expiry report
- Serial number report

Purchase Reports

- Purchase by supplier
- Purchase by branch
- Purchase by item
- Purchase return
- Supplier balance
- Supplier aging

Customer Reports

- Customer ledger
- Customer balance
- Customer aging
- Customer sales
- Customer payment
- Customer returns

Financial Reports

- Trial Balance
- General Ledger
- Profit & Loss
- Balance Sheet
- Cash Flow
- Accounts Receivable
- Accounts Payable
- Expense Report
- Tax Report

Every report should support:

- Date range
- Company
- Branch
- Warehouse
- Customer
- Supplier
- Item
- Category

And allow:

- Search
- Sorting
- Filtering

- Pagination
 - Excel export
 - CSV export
 - PDF
 - Print
-

38. DASHBOARD

Create an executive dashboard.

Show:

- Today's sales
- Today's purchases
- Today's profit
- Cash balance
- Bank balance
- Receivables
- Payables
- Stock value
- Low stock items
- Out-of-stock items
- Top-selling items
- Top customers
- Top suppliers
- Sales by branch
- Sales trend
- Profit trend

Dashboard must respect user permissions and branch/company access.

39. AUDIT TRAIL

Every important action should be logged.

Audit fields:

- User
- Date/time
- Company
- Branch
- Module
- Action

- Record ID
- Old value
- New value
- IP
- Device/session

Log:

- Login
- Logout
- Create
- Update
- Delete
- Void
- Approve
- Reverse
- Price override
- Discount override
- Stock adjustment
- Accounting changes

40. DOCUMENT WORKFLOW

Use statuses.

Example Sales Invoice:

DRAFT

→ SUBMITTED

→ APPROVED

→ POSTED

→ PAID

or

→ PARTIALLY PAID

→ CANCELLED

→ RETURNED

Do not allow arbitrary database modification of posted financial transactions.

Use reversal/correction transactions instead.

41. DATABASE DESIGN

Design a normalized relational database.

Create proper tables for:

- companies
- branches
- warehouses
- users
- roles
- permissions
- role_permissions
- user_roles
- user_branches
- menus
- categories
- brands
- units
- items
- item_barcodes
- item_prices
- customer_prices
- customers
- suppliers
- sales
- sale_lines
- sale_payments
- sales_returns
- purchase_orders
- purchases
- purchase_lines
- purchase_returns
- inventory_transactions
- stock_balances
- stock_transfers
- stock_transfer_lines
- batches
- serial_numbers
- chart_of_accounts
- journal_entries

- journal_lines
- payments
- receipts
- expenses
- cash_registers
- cash_sessions
- tax_codes
- discounts
- audit_logs
- document_sequences
- system_settings

Add any additional tables necessary for a proper enterprise system.

42. DATABASE PRINCIPLES

Use:

- Primary keys
- Foreign keys
- Unique constraints
- Check constraints where appropriate
- Indexes
- CreatedAt
- CreatedBy
- UpdatedAt
- UpdatedBy
- Soft delete where appropriate

Do not physically delete financial transactions.

Use status fields and reversal/cancellation mechanisms.

Optimize indexes for:

- Barcode search
 - SKU search
 - Invoice number
 - Customer
 - Supplier
 - Branch
 - Company
 - Date ranges
 - Inventory transactions
-

43. TRANSACTION SAFETY

POS transactions must be atomic.

When completing a sale:

1. Validate items
2. Validate stock
3. Calculate pricing
4. Calculate discount
5. Calculate tax
6. Calculate total
7. Save invoice
8. Save invoice lines
9. Save payment
10. Update inventory
11. Create accounting entries
12. Update customer balance if credit
13. Update cashier session
14. Commit transaction

If anything fails, rollback the complete transaction.

Never leave partially completed financial transactions.

44. CONCURRENCY

The application must handle multiple users selling the same inventory simultaneously.

Protect against:

- Duplicate invoice numbers
- Overselling stock
- Duplicate payments
- Duplicate journal entries
- Race conditions
- Double submission

Use:

- Database transactions
- Appropriate isolation
- Row locking where necessary
- Unique constraints

- Idempotency where appropriate
-

45. OFFLINE POS ARCHITECTURE

Design the POS so that offline capability can be added.

Consider:

- Local browser storage
- IndexedDB
- Local transaction queue
- Synchronization
- Conflict resolution

The initial implementation may be online-first, but the architecture should not prevent future offline operation.

46. HARDWARE / INTEGRATION ARCHITECTURE

Design integration points for:

- Barcode scanners
- Thermal printers
- A4 printers
- Cash drawers
- Customer displays
- Weighing scales
- Barcode label printers

Also design APIs for:

- E-commerce
- Mobile applications
- External accounting systems
- Payment gateways
- Government tax systems
- ERP integrations

Use an integration/service abstraction instead of hard-coding external vendors.

47. API DESIGN

Create clean APIs.

Example:

Authentication:

POST /api/auth/login

POST /api/auth/logout

Users:

GET /api/users

POST /api/users

PUT /api/users/:id

Sales:

GET /api/sales

POST /api/sales

POST /api/sales/:id/approve

POST /api/sales/:id/void

POST /api/sales/:id/return

Inventory:

GET /api/inventory

GET /api/inventory/ledger

POST /api/inventory/adjustment

POST /api/inventory/transfer

Customers:

GET /api/customers

POST /api/customers

GET /api/customers/:id/ledger

Reports:

GET /api/reports/sales

GET /api/reports/inventory

GET /api/reports/profit-loss

Follow RESTful and consistent API conventions.

48. SECURITY

Implement enterprise security.

Requirements:

- Password hashing
- Secure sessions
- CSRF protection where applicable
- XSS protection
- SQL injection protection
- Input validation
- API authorization
- Rate limiting
- Secure cookies
- Security headers
- Audit logging
- Permission checks server-side
- Never trust frontend permissions
- Validate all financial transactions server-side

Use a validation library such as Zod or equivalent.

49. UI/UX REQUIREMENTS

The application must look like a modern commercial ERP/POS system.

Design principles:

- Clean
- Professional
- Fast
- Minimal clicks
- Responsive
- Desktop-first for POS
- Mobile/tablet-friendly for management
- Accessible
- Consistent spacing
- Consistent typography
- Reusable components

Use Tailwind CSS.

Create:

- Sidebar navigation
- Top navigation
- Breadcrumbs
- Cards
- Tables
- Forms
- Modals
- Drawers
- Tabs
- Dropdowns
- Toast notifications
- Confirmation dialogs
- Loading states
- Empty states
- Error states

50. POS UX

The POS must prioritize speed.

A cashier should be able to:

1. Scan barcode
2. Add item
3. Change quantity
4. Apply permitted discount
5. Select customer

6. Select payment
7. Complete sale
8. Print receipt

with minimal mouse interaction.

Use large buttons and clear totals.

Support touch screens.

51. RESPONSIVE DESIGN

The application must support:

- Desktop
- Laptop
- Tablet
- POS terminal

Management pages should adapt to smaller screens.

The POS itself should be optimized for:

- 1024×768
 - 1366×768
 - 1920×1080
-

52. ERROR HANDLING

Create consistent error handling.

Examples:

"Insufficient stock."

"Customer credit limit exceeded."

"You do not have permission to apply this discount."

"Invoice has already been posted."

"Cannot modify a posted transaction."

"Payment amount exceeds invoice balance."

"Barcode already exists."

Use user-friendly messages while logging technical details server-side.

53. CONFIGURATION / SYSTEM SETTINGS

Create a central settings module.

Settings should include:

Company

- Company information
- Currency
- Tax

POS

- Default price level
- Receipt printer
- Invoice format
- Auto-print
- Barcode behavior
- Negative stock policy

Inventory

- Cost method
- Negative stock
- Reorder settings
- Batch tracking
- Serial tracking

Accounting

- Fiscal year
- Default accounts
- Posting rules

Security

- Password policy
- Session timeout
- Login attempts

54. GENERIC BUSINESS CONFIGURATION

Do not hard-code business rules that only apply to one industry.

For example:

A retail store may need simple inventory.

A pharmacy may need:

- Batch
- Expiry
- FEFO

An electronics store may need:

- Serial numbers
- IMEI
- Warranty

A wholesale business may need:

- Customer price lists
- Quantity pricing
- Credit limits

Design the system using configurable features.

55. MASTER DATA

Create master-data management for:

- Company
- Branch
- Warehouse
- Users
- Roles
- Permissions
- Customers
- Suppliers
- Items
- Categories
- Brands

- Units
 - Taxes
 - Price levels
 - Payment methods
 - Sale types
 - Expense types
 - Account types
 - Currencies
 - Document types
 - Number sequences
-

56. IMPORT / EXPORT

Allow bulk import through CSV/Excel.

Examples:

- Items
- Customers
- Suppliers
- Opening stock
- Opening balances
- Price lists

Provide validation before import.

Show:

- Successful records
- Failed records
- Error reason
- Row number

Do not partially import invalid financial data.

57. OPENING BALANCES

Provide initial setup for:

- Customer opening balances
- Supplier opening balances
- Cash opening
- Bank opening

- Inventory opening
- General ledger opening balances

Opening balances must generate appropriate accounting entries.

58. FINANCIAL YEAR

Support:

- Fiscal year
- Accounting periods
- Period opening
- Period closing
- Locked periods

Prevent users from posting transactions into closed accounting periods unless explicitly authorized.

59. APPROVAL SYSTEM

Create configurable approval workflows.

Examples:

Purchase > 500,000 requires manager approval.

Discount > 10% requires supervisor approval.

Stock adjustment > 100 units requires manager approval.

Credit sale above credit limit requires manager approval.

Make approval rules configurable.

60. NOTIFICATIONS

Create notification architecture.

Notifications for:

- Low stock
- Out of stock

- Reorder
 - Expiry
 - Credit limit exceeded
 - Overdue customer
 - Overdue supplier
 - Pending approval
 - Cash variance
 - Failed integrations
-

61. SEARCH

Implement global and module-specific search.

Search by:

- Barcode
- SKU
- Item name
- Invoice number
- Customer
- Supplier
- Phone
- Account code
- Transaction ID

Barcode search must be extremely fast.

62. PERFORMANCE

Design for large datasets.

Assume:

- Millions of inventory transactions
- Millions of sales lines
- Thousands of products
- Hundreds of branches
- Hundreds/thousands of users

Use:

- Pagination
- Server-side filtering

- Proper indexing
- Caching where useful
- Efficient queries
- Lazy loading
- Background jobs where appropriate

Never load millions of records into the browser.

63. BACKGROUND JOBS

Design support for background jobs for:

- Report generation
 - Email invoices
 - Data synchronization
 - Stock calculations
 - Notifications
 - Scheduled backups
 - Integration synchronization
-

64. LOGGING AND MONITORING

Implement structured application logging.

Log:

- Errors
- API requests
- Authentication failures
- Important business events
- Integration errors

Do not log passwords, tokens, or sensitive credentials.

65. TESTING

Create tests for critical business logic.

At minimum:

Unit tests

- Pricing
- Discounts
- Tax
- Inventory calculations
- Accounting calculations
- Credit limits

Integration tests

- Complete sale
- Complete purchase
- Return
- Stock transfer
- Payment
- Accounting posting

Authorization tests

Verify that users cannot access unauthorized:

- Companies
- Branches
- Warehouses
- Modules
- Transactions

End-to-end tests

Test the main POS workflow.

66. SEED DATA

Create realistic seed data.

Example:

Company:

"Demo Trading Company"

Branches:

- Main Branch
- City Branch

- Warehouse Branch

Users:

- Administrator
- Manager
- Cashier
- Inventory Officer
- Accountant

Products:

- Sample products
- Categories
- Brands
- Units
- Price levels

Customers:

- Walk-in Customer
- Retail Customer
- Wholesale Customer

Suppliers:

- Sample suppliers

Accounts:

Complete sample Chart of Accounts.

67. PROJECT STRUCTURE

Provide a clean project structure.

For example:

`/app /components /features /modules /lib /services /api /db /types /hooks /utils /config /public /tests`

Separate:

- UI
- Domain/business logic
- Database
- API

- Authentication
- Authorization
- Reporting

Do not put all logic inside React components.

68. DOMAIN-DRIVEN MODULE STRUCTURE

Where practical, organize functionality around business domains:

/modules/auth

/modules/users

/modules/companies

/modules/branches

/modules/products

/modules/customers

/modules/suppliers

/modules/sales

/modules/purchases

/modules/inventory

/modules/accounting

/modules/reports

/modules/settings

Each module should have clear responsibilities.

69. ACCOUNTING POSTING ENGINE

Create a reusable accounting posting engine.

Instead of manually writing accounting logic throughout the application, create a service such as:

AccountingPostingService

It should receive a business transaction and generate balanced journal entries.

Example:

SalePostingService

PurchasePostingService

PaymentPostingService

ReceiptPostingService

InventoryAdjustmentPostingService

StockTransferPostingService

Every posting must verify:

TOTAL DEBITS = TOTAL CREDITS

70. INVENTORY ENGINE

Create a centralized inventory service.

Example:

InventoryService.sale()

InventoryService.purchase()

InventoryService.return()

InventoryService.adjust()

InventoryService.transfer()

InventoryService.reserve()

InventoryService.release()

This prevents inventory logic from being duplicated across the application.

71. PRICING ENGINE

Create a centralized pricing engine.

Example:

```
PricingService.getPrice({ item, customer, quantity, branch, date, priceLevel })
```

It should determine:

1. Customer-specific price
2. Customer price level
3. Quantity price
4. Promotional price
5. Standard price

according to configurable priority.

72. BUSINESS RULE ENGINE

Important business rules must be centralized.

Examples:

- Can user sell below minimum price?
- Can stock go negative?
- Can customer exceed credit limit?
- Can cashier apply discount?
- Can posted invoice be edited?
- Can transaction be posted in closed period?

Do not scatter these rules throughout the UI.

73. DATA OWNERSHIP

Every business transaction should have appropriate ownership fields:

- Company ID
- Branch ID

- Warehouse ID where applicable

Queries must automatically enforce the logged-in user's scope.

Example:

A Branch 1 cashier must never be able to access Branch 2 sales simply by modifying an API request.

Authorization must be enforced server-side.

74. SOFT DELETE

Use soft deletion for master data where appropriate.

Examples:

- Customer
- Supplier
- Product
- Category
- User

Do not delete historical transaction records.

75. REPORT FILTER ARCHITECTURE

Create reusable report filters.

Common filters:

- From date
- To date
- Company
- Branch
- Warehouse
- Customer
- Supplier
- Item
- Category
- Brand
- User
- Payment method

Remember selected filters.

Allow saving favorite report filters.

76. PRINTING

Support:

Thermal

58mm

80mm

A4

Professional invoice layout.

Allow configurable templates.

Include:

- Company logo
 - Company details
 - Branch
 - Invoice number
 - Customer
 - Cashier
 - Items
 - Discounts
 - Tax
 - Total
 - Payment
 - Footer
-

77. INTERNATIONALIZATION

Architect for multiple languages.

Use translation keys rather than hard-coded UI text.

Support:

- English initially
 - RTL capability for future languages
 - Currency formatting
 - Date formatting
 - Number formatting
-

78. API DOCUMENTATION

Generate API documentation using OpenAPI/Swagger or equivalent.

Document:

- Authentication
 - Request
 - Response
 - Errors
 - Authorization requirements
-

79. DEPLOYMENT

Provide deployment architecture for:

Development

Local:

Next.js

Node.js

PostgreSQL

Production

Possible architecture:

Browser

↓

Reverse Proxy

↓

Next.js Frontend

↓

Node.js API

↓

PostgreSQL

Also explain options for:

- Docker
- Docker Compose
- Linux server
- Cloud deployment
- Database backup
- Environment variables

80. ENVIRONMENT CONFIGURATION

Use environment variables for:

- Database URL
- JWT/session secrets
- Email configuration
- Storage
- External APIs
- Payment integrations

Never commit secrets.

Provide:

.env.example

81. BACKUP & RECOVERY

Design database backup strategy.

Support:

- Automated backup
 - Backup retention
 - Restore procedure
 - Disaster recovery considerations
-

82. IMPORTANT UX REQUIREMENT — DO NOT OVERENGINEER THE FIRST SCREEN

The POS cashier interface must remain simple even though the backend is powerful.

A cashier should see only what they need.

Management users can access advanced functionality.

Use role-based UI visibility.

83. ADMINISTRATOR EXPERIENCE

Administrator should have access to:

- Company setup
- Branch setup
- Users
- Roles
- Permissions
- Menus
- Tax
- Pricing
- Inventory settings
- Accounting settings
- POS settings
- Document numbering
- Approval workflows
- Integrations
- System configuration

84. DASHBOARD DRILL-DOWN

Dashboard metrics should be clickable.

Example:

"Today's Sales: 1,250,000"

Click → sales report filtered for today.

"Low Stock: 27"

Click → low-stock report.

"Receivables: 4,500,000"

Click → customer aging.

85. GENERIC POS TERMINAL MANAGEMENT

Add POS terminal management.

Fields:

- Terminal ID
- Terminal name
- Branch
- Cash drawer
- Printer
- Assigned cashier
- Status

Support:

- Open terminal
 - Close terminal
 - Terminal sales
 - Terminal reconciliation
-

86. WALK-IN CUSTOMER

Create a configurable default:

"Walk-in Customer"

Cashiers can complete cash sales without entering full customer information.

Credit sales should require an identified customer unless configured otherwise.

87. CUSTOMER CREDIT CONTROL

Implement:

- Credit limit
- Credit days
- Current outstanding
- Overdue amount
- Available credit

Before completing credit sale:

Check:

Current Outstanding + New Invoice $\leq$ Credit Limit

If exceeded:

- Block
- Warn
- Or require approval

according to configuration.

88. STOCK RESERVATION

Design optional stock reservation for:

- Orders
- Quotations
- Pending sales
- Online orders

Reserved stock must not be incorrectly shown as freely available stock.

89. QUOTATIONS / ORDERS

Include optional:

- Sales quotation
- Sales order
- Convert quotation → invoice
- Convert order → invoice

Likewise:

- Purchase requisition
 - Purchase order
 - Goods receipt
 - Purchase invoice
-

90. EXPENSE MANAGEMENT

Support expenses such as:

- Rent
- Electricity
- Transport
- Salaries
- Utilities
- Miscellaneous

Expenses should be posted to appropriate expense accounts.

91. GENERAL LEDGER

The ledger must support:

- Account
- Date
- Reference
- Debit
- Credit
- Balance

- Company
- Branch

Allow drill-down from ledger transaction to original document.

92. TRACEABILITY

Every accounting transaction should be traceable.

Example:

General Ledger

→ Journal Entry

→ Sales Invoice

→ Customer

→ Invoice Lines

→ Inventory Transactions

This should work in both directions where practical.

93. DATA CONSISTENCY

The system must maintain consistency between:

Sales

Inventory

Customers

Cash

Accounting

Tax

All modules must use transactional operations.

Do not allow sales to complete if accounting/inventory posting fails unless an explicit asynchronous architecture is implemented with reliable reconciliation.

94. DESIGN SYSTEM

Create a reusable Tailwind design system.

Define:

- Typography
- Spacing
- Buttons
- Inputs
- Selects
- Tables
- Modals
- Alerts
- Badges
- Cards
- Tabs
- Dropdowns

Do not create every page with completely different styling.

95. IMPORTANT IMPLEMENTATION RULE

Do not generate the entire project blindly in one response.

First provide:

PHASE 1 — ARCHITECTURE

1. Overall architecture
2. Module architecture
3. Database architecture
4. Entity relationship design
5. Authorization architecture
6. Accounting architecture
7. Inventory architecture
8. Pricing architecture
9. API architecture
10. Frontend architecture
11. Folder structure

12. Deployment architecture

Then wait for approval.

After approval, implement incrementally.

96. IMPLEMENTATION PHASES

Use the following development order:

Phase 1

Project foundation

- Next.js
- Node.js
- TypeScript
- Tailwind
- Database
- ORM
- Authentication

Phase 2

Security

- Users
- Roles
- Permissions
- Company
- Branch
- Menu authorization

Phase 3

Master data

- Items
- Categories
- Brands
- Units
- Customers
- Suppliers
- Taxes
- Price levels

Phase 4

Inventory

- Opening stock
- Stock ledger
- Adjustments
- Transfers
- Reorder

Phase 5

Sales

- POS
- Barcode
- Pricing
- Discounts
- Customers
- Cash sales
- Credit sales
- Payments
- Returns

Phase 6

Purchasing

- Purchase orders
- Purchase invoices
- Supplier payments
- Purchase returns

Phase 7

Accounting

- Chart of accounts
- Journal
- Ledger
- AR
- AP
- Cash
- Bank
- Automatic postings

Phase 8

Reports

Phase 9

Approvals

Phase 10

Integrations

Phase 11

Testing

Phase 12

Deployment

97. WHAT I EXPECT FROM YOU

For every implementation phase:

1. Explain the architecture briefly.
2. Show the database schema.
3. Show migrations.
4. Show TypeScript types/interfaces.
5. Show backend services.
6. Show API endpoints.
7. Show validation.
8. Show authorization.
9. Show frontend components.
10. Show pages.
11. Show tests.
12. Explain how the components interact.
13. Identify potential problems.
14. Suggest improvements.

When generating code, provide complete working code rather than pseudocode whenever practical.

Do not leave critical functionality as:

"TODO"

"Implement later"

"Add your code here"

For complex functionality, implement it incrementally.

98. CODE QUALITY

Follow:

- SOLID principles
- DRY
- Clean Architecture where appropriate
- Separation of concerns
- Strong typing
- Reusable components
- Service layer
- Repository/data-access abstraction where appropriate
- Centralized validation
- Centralized authorization
- Centralized business rules

Avoid:

- Huge components
 - Duplicate logic
 - Business logic inside UI components
 - Direct database access from random UI files
 - Hard-coded permissions
 - Hard-coded company IDs
 - Hard-coded branch IDs
 - Hard-coded prices
 - Hard-coded tax rates
 - Hard-coded account IDs
-

99. CRITICAL ACCOUNTING REQUIREMENT

Do not treat accounting as simply another CRUD module.

Accounting must be designed around **double-entry bookkeeping**.

Every posted financial transaction must produce balanced accounting entries.

Examples:

Sale

Purchase

Customer payment

Supplier payment

Expense

Sales return

Purchase return

Stock adjustment

Cash transfer

Bank transaction

All must have proper accounting treatment.

100. CRITICAL INVENTORY REQUIREMENT

Do not store only a simple "stock quantity" field and assume that is sufficient.

Maintain a proper inventory transaction history.

Stock should be derivable from transactions and/or a controlled stock balance table.

The system must be able to answer:

"Why is this item showing 127 units?"

The answer should be traceable through:

Opening stock

- Purchases
- Purchase returns

- Sales

- Sales returns

+/- Adjustments

+/- Transfers

= Current stock

101. CRITICAL MULTI-BRANCH REQUIREMENT

Every relevant transaction must maintain company and branch context.

Reports must support:

- Current branch
- Selected branch
- All branches
- Company consolidated

Do not duplicate the entire product/customer master unnecessarily per branch.

Use shared master data with branch-specific configuration/pricing/stock where appropriate.

102. CRITICAL SECURITY REQUIREMENT

Never rely on frontend hiding buttons as authorization.

Even if the UI hides:

"Delete"

"Discount"

"View other branch"

the backend must independently verify authorization.

A malicious API request must still be rejected.

103. FINAL DELIVERABLES

Ultimately produce:

1. Complete architecture document
 2. ER diagram
 3. Database schema
 4. Migration scripts
 5. Seed data
 6. Authentication
 7. Authorization
 8. Company/branch management
 9. Master data
 10. POS
 11. Sales
 12. Purchasing
 13. Inventory
 14. Customers
 15. Suppliers
 16. Pricing
 17. Discounts
 18. Barcode
 19. Accounting
 20. Reports
 21. Audit logging
 22. Configuration
 23. Tests
 24. API documentation
 25. Docker configuration
 26. Deployment documentation
 27. Database backup strategy
 28. User manual/basic setup documentation
-

104. FIRST RESPONSE REQUIREMENT

Do NOT immediately start generating thousands of lines of code.

Your first response should contain:

A. Recommended architecture

Show the complete architecture diagram in text/ASCII.

B. Technology decisions

Explain:

- Next.js architecture
- Node.js architecture
- TypeScript
- Tailwind
- ORM
- PostgreSQL
- Authentication
- Authorization
- Validation
- Testing

C. Module breakdown

Show all modules and dependencies.

D. Database architecture

Show the major entities and relationships.

E. Accounting architecture

Explain how sales/purchases/inventory interact with accounting.

F. Inventory architecture

Explain stock calculation and transaction flow.

G. Multi-company/multi-branch architecture

Explain data isolation and reporting.

H. Permission architecture

Explain RBAC and branch/company permissions.

I. Folder structure

Show the proposed project structure.

J. Development roadmap

Break the project into practical implementation milestones.

K. Risks and recommendations

Identify architectural risks before coding.

After presenting this architecture, WAIT for my approval before proceeding with implementation.

105. MOST IMPORTANT DESIGN PRINCIPLE

Think of this application as a **small ERP system with a high-speed POS front end**, not merely a billing screen.

The POS is only one part of the system.

The underlying architecture must correctly connect:

POS

↓

Sales

↓

Inventory

↓

Customers

↓

Payments

↓

Cash/Bank

↓

Accounts Receivable



General Ledger



Financial Reports

Similarly:

Purchasing



Inventory



Suppliers



Payments



Accounts Payable



General Ledger

And:

Company



Branches



Warehouses

↓

Users

↓

Permissions

↓

Transactions

↓

Branch/Company Reports

The final design must maintain consistency across all these domains.

Start with the architecture and wait for approval before writing the implementation.